

Security Audit

PING (Gateway)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	41
Conclusion	42
Our Methodology	43
Disclaimers	45
About Hashlock	46

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The PING team partnered with Hashlock to conduct a security audit of their MyToken.sol, PINGLock.sol, PINGLockInterface.sol smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

PING is a blockchain-powered gaming platform that aims to revolutionize digital play by merging immersive game experiences with decentralized finance. The platform leverages NFT technology and tokenized rewards to empower players, allowing them to earn digital assets with real-world value through gameplay. With a focus on transparency and community engagement, PING creates a thriving in-game economy where users actively shape the platform's future, blending the worlds of entertainment and blockchain innovation seamlessly.

Project Name: PING

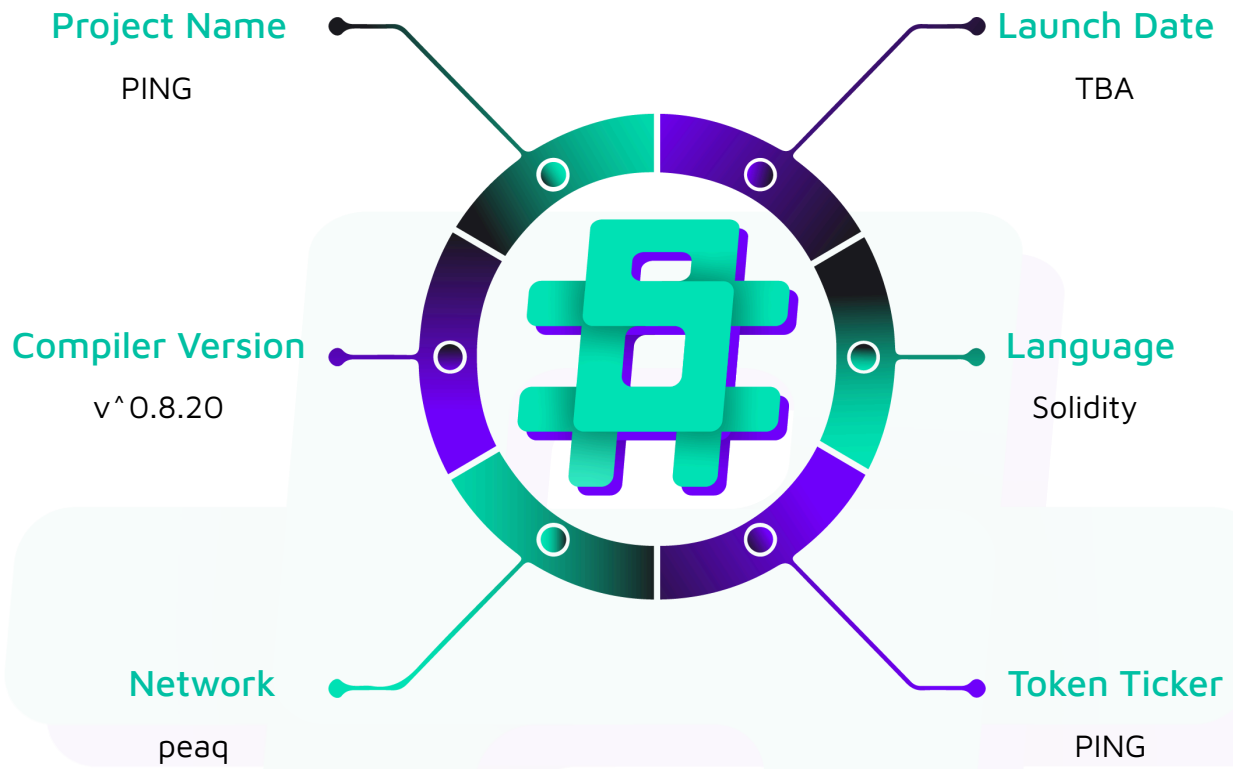
Project Type: Payment Gateway

Compiler Version: ^0.8.20

Website: <https://pingame.io/>

Logo:



Visualised Context:

Project Visuals:

Earn in Gaming Earn in Computing

The DePIN infrastructure for building, gaming,
and computing.

PRE-ORDER DEPIN DEVICE

DePIN

GAME

DePIN

Using DePIN technology, gamers can contribute their laptop computing power to the network for rewards, creating an affordable resource pool for game developers to access rendering and AI training services.

Game

PINGame is a Web3 gaming platform. We aim to unite the gaming community and build a sustainable P2E gaming ecosystem that empowers all type of games. Everyone is invited to play, create and earn in clicks.

Audit Scope

We at Hashlock audited the solidity code within the PING project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	PING Protocol Smart Contracts
Platform	peaq / Solidity
Audit Date	February, 2025
Contract 1	MyToken.sol
Contract 1 MD5 Hash	4602d4d96d8965bc4ff68d4c629d1b12
Contract 2	PinGameLock.sol
Contract 2 MD5 Hash	d09369057a52ffffd097e9fdd976d35b
Contract 3	PingameLockInterface.sol
Contract 3 MD5 Hash	a5b82d771b080674d992f8123068ee29
GitHub Commit Hash	0a05f23ee1560d0d177109da4039acfd44aef674

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

3 Medium severity vulnerabilities

13 Low severity vulnerabilities

3 Gas Optimisations

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
MyToken.sol - ERC20 token that is used for tests	Contract achieves this functionality.
PinGameLock.sol - Allows users to: - Deposit supported tokens for access rights - Use discounts when available - Receive refunds when processed by admin - Allows admins to: - Add/remove supported tokens - Set token prices and discounts - Set discount validity periods - Process refunds to users - Withdraw collected tokens - Pause/unpause contract operations	Contract achieves this functionality.
PingameLockInterface.sol - Interface contract for PinGameLock.sol	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the PING project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the PING project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] PinGameLock#setDiscount - No checks done to make sure discount is not greater than the price of the token leads to bricking the deposit function

Description

The setDiscount function allows the owner to add a discount to the discount arrays of a token. The deposit function has calculations for the actualPrice of the token by subtracting the discount from the price of this token. As there are no checks done to make sure that the discount can not exceed the price of a token, if the discount amount is set higher than the price, all calls to the deposit function will revert due to underflow. This vulnerability also breaks the prices view function.

Vulnerability Details

The setDiscount function allows the owner to add a discount to the discount arrays of a token.

```
function setDiscount(address _token,uint256 discount_price,uint256 discount_amount)
onlyOwner external {

    require(discount_price > 0,"price should be greater than 0");

    require(discount_amount >= 0,"amount should be greater than 0");

    require(supportedTokens[_token],"token not exist");

    // sort the discounts array descending order

    discountsAmounts[_token].push(discount_price);

    // console.log(discountsAmounts[_token][0]);

    uint256[] memory sortedAmounts =
Arrays.sort(discountsAmounts[_token],Comparators.gt);

    discountsAmounts[_token] = sortedAmounts;
```

```

discounts[_token][discount_price] = discount_amount;

updateCurrentDiscount(_token);

emit SetDiscount(_token, discount_price, discount_amount);

}

```

This function has checks to make sure that the discount price and discount amounts are greater than 0 and the token inputted is supported. This function will then add a new discount to this tokens discounts array and sort this array to make the highest discount the first item in this array. However, there are no checks implemented to make sure that the added discount is less than the price of the token.

In the deposit function, the actualPrice of the token is calculated by subtracting the discount from the price of this token.

```

function deposit( address _token,uint256 amount,string memory refererCode) external {

    require(supportedTokens[_token],"token not supported");

    require(amount > 0,"amount should be greater than 0");

    require(originPrices[_token] > 0,"price should be greater than 0");

    uint256 cd = getCurrentDiscount(_token);

    uint256 actualPrice = originPrices[_token] - cd;

    // rest of the function
}

```

If this tokens discount is higher than the price, this calculation will cause a revert due to an underflow. As there are no ways of removing discounts from the discount array, the only fix to this problem is updating the price of the token to be higher than the discount but this fix is a workaround fix that breaks the integrity of the contract. This vulnerability also breaks the prices view function.

Proof of Concept

- 1) Set up a Foundry test suite

```

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.20;

```



```

import "forge-std/Test.sol";

import "../contracts/PinGameLock.sol";

import "../contracts/MyToken.sol";

contract PinGameLockTest is Test {

    PinGameLock public lock;

    MyToken public token;

    address owner = address(1);

    address user = address(2);

    address user2 = address(3);

    function setUp() public {

        // Deploy test token

        token = new MyToken("Test Token", "TEST");

        token.mint(user, 1000 ether);

        token.mint(user2, 1000 ether);

        // Deploy PinGameLock

        lock = new PinGameLock();

        address[] memory tokens = new address[](1);

        tokens[0] = address(token);

        uint256[] memory prices = new uint256[](1);

        prices[0] = 100 ether;

        // Initialize with owner and supported tokens

```

```

lock.initialize(owner, prices, tokens);

// Set discount valid time

vm.prank(owner);

lock.setDiscountValidTo(block.timestamp + 10 days);

// Approve tokens

vm.prank(user);

token.approve(address(lock), type(uint256).max);

vm.prank(user2);

token.approve(address(lock), type(uint256).max);

}

```

2) Implement and run the following test to observe the vulnerability.

```

function testUnderflowDiscountUpdate() public {

    vm.startPrank(owner);

    // Setting discount higher than price - this should succeed

    lock.setDiscount(address(token), 110 ether, 1);

    vm.stopPrank();

    vm.startPrank(user);

    // Now try to deposit - this should revert

    vm.expectRevert(stdError.arithmeticError);

    lock.deposit(address(token), 100 ether, "TEST");

    vm.stopPrank();
}

```



```

    vm.startPrank(owner);

    // Setting discount lower does not fix

    lock.setDiscount(address(token), 20 ether, 1);

    vm.stopPrank();

    vm.startPrank(owner);

    // prices function is also broken

    vm.expectRevert(stdError.arithmeticError);

    uint256 price = lock.prices(address(token));

    vm.assertEq(price, 0);

    vm.stopPrank();

    vm.startPrank(user);

    // Now try to deposit again - this should revert

    vm.expectRevert(stdError.arithmeticError);

    lock.deposit(address(token), 100 ether, "TEST");

    vm.stopPrank();

}

```

Impact

The deposit function that is integral for this contract's functionality will be bricked permanently and will not work.

Recommendation

Implement checks in the function to make sure that the discount can not be greater than the price of a token.

Status

Resolved



[H-02] PinGameLock#setPrice - No checks done to make sure that the new price of a token is greater than the current discount of this token leads to bricking the `deposit` function

Description

The `setPrice` function allows the owner to set the price of a token. However, similar to [H-01], this function lacks checks to make sure that the price will stay higher than the discount. When the price of a token is set to a lower amount than the current discount, it will brick the `deposit` function as the `actualPrice` calculation will always revert due to an `underflow`. This vulnerability also breaks the `prices` view function.

Vulnerability Details

The `setPrice` function allows the owner to set the price of a token.

```
function setPrice(address _token,uint256 _price) onlyOwner external {  
  
    require(supportedTokens[_token],"token not exist");  
  
    uint256 oldPrice = originPrices[_token];  
  
    originPrices[_token] = _price;  
  
    emit SetPrice(_token, oldPrice, _price);  
  
}
```

As observed, the only check done in this function is to make sure that the token inputted is a supported token. This means that the price of a token can be set to an amount that is lower than the discount.

In the `deposit` function, the `actualPrice` of the token is calculated by subtracting the discount from the price of this token.

```
function deposit( address _token,uint256 amount,string memory refererCode) external {  
  
    require(supportedTokens[_token],"token not supported");  
  
    require(amount > 0,"amount should be greater than 0");  
  
}
```

```

require(originPrices[_token] > 0, "price should be greater than 0");

uint256 cd = getCurrentDiscount(_token);

uint256 actualPrice = originPrices[_token] - cd;

// rest of the function

```

If this tokens discount is higher than the price, this calculation will cause a revert due to an underflow. As there are no ways of removing discounts from the discount array, the only fix to this problem is updating the price of the token to be higher than the discount but this fix is a workaround fix that breaks the integrality of the contract. This vulnerability also breaks the prices view function.

Proof of Concept

- 1) Set up the test suite as shown in [H-01]
- 2) Implement the following test and run it to observe the vulnerability

```

function testUnderflowPriceUpdate() public {

    vm.startPrank(owner);

    lock.setDiscount(address(token), 90 ether, 2);

    vm.stopPrank();

    vm.startPrank(user);

    // Try to deposit, it should work

    lock.deposit(address(token), 100 ether, "TEST");

    vm.stopPrank();

    vm.startPrank(owner);

    // Setting price lower than existing discount

    lock.setPrice(address(token), 50 ether);

    vm.stopPrank();

```

```

    vm.startPrank(user2);

    // Now try to deposit - this should revert due to underflow in price calculation

    vm.expectRevert(stdError.arithmeticError);

    lock.deposit(address(token), 100 ether, "TEST");

    vm.stopPrank();
}

```

Impact

The `deposit` function that is integral for this contract's functionality will be bricked permanently and will not work.

Recommendation

Implement checks in the function to make sure that the new price of the token can not be lower than the existing discount.

Status

Resolved

[H-03] PinGameLock#deposit - Discounts do not apply and users spend the full amount even when there are valid discounts

Description

The `deposit` function allows users to deposit tokens into the contract. This function has a discount system implemented but this discount does not change the amount that users have to spend. Even with valid discounts, users will spend the full amount and their deposited balance will update with this amount.

Vulnerability Details

The `deposit` function allows users to deposit tokens into the contract

```
function deposit( address _token,uint256 amount,string memory refererCode) external {
```

```

require(supportedTokens[_token], "token not supported");

require(amount > 0, "amount should be greater than 0");

require(originPrices[_token] > 0, "price should be greater than 0");

uint256 cd = getCurrentDiscount(_token);

uint256 actualPrice = originPrices[_token] - cd;

require(amount >= actualPrice, "should be greater than price");

require(userBought[msg.sender] == false, "already bought");

//get discount

SafeERC20.safeTransferFrom(IERC20(_token), msg.sender, address(this), amount);

userBalance[msg.sender][_token] += amount;

userBought[msg.sender] = true;

if (cd > 0 && amount < originPrices[_token]){ // user used the discount

    discounts[_token][cd] -= 1;

    updateCurrentDiscount(_token);

}

emit Deposit(msg.sender, _token, amount, refererCode);

}

```

As observed in the part highlighted in yellow, there are calculations done to apply a discount and calculate the actual price of the token. However, even with this calculation, the full amount entered by the user is transferred from the user as seen in the part highlighted in green. This means that the discounts are not implemented correctly and users will still spend the full amount they input, their deposited balance

will also increase by this amount. The only way for users to make use of the discount is to know the price and the current discount and input the correct amount.

Proof of Concept

- 1) Set up the test suite as shown in [H-01]
- 2) Implement the following test and run it to observe the vulnerability.

```
function testDiscountDoesNotApply() public {  
  
    vm.startPrank(owner);  
  
    // Setting two discounts  
  
    lock.setDiscount(address(token), 90 ether, 1);  
  
    lock.setDiscount(address(token), 80 ether, 1);  
  
    vm.stopPrank();  
  
  
    uint256 user1BalanceBefore = token.balanceOf(user);  
  
    // Deposit with user1  
  
    vm.startPrank(user);  
  
    lock.deposit(address(token), 100 ether, "TEST");  
  
    vm.stopPrank();  
  
    uint256 user1BalanceAfter = token.balanceOf(user);  
  
  
    uint256 user2BalanceBefore = token.balanceOf(user2);  
  
    // Deposit with user2  
  
    vm.startPrank(user2);  
  
    lock.deposit(address(token), 99 ether, "TEST");  
  
    vm.stopPrank();  
  
    uint256 user2BalanceAfter = token.balanceOf(user2);  
  
}
```

```
uint256 user1Deposited = lock.userBalance(user, address(token));

uint256 user2Deposited = lock.userBalance(user2, address(token));


//assertions to make sure both users deposited the correct amount

assertEq(user1Deposited, 100 ether);

assertEq(user2Deposited, 99 ether);


// assertions prove both users spent the full amount

assertEq(user1BalanceBefore - user1BalanceAfter, 100 ether);

assertEq(user2BalanceBefore - user2BalanceAfter, 99 ether);

}
```

Impact

The discount functionality is not implemented correctly. Even with valid discounts present, users still pay the full amount.

Recommendation

Change the function to take discounts into consideration and correctly calculate the amount of tokens users must send to the contract.

Alternatively, do not take user input for the amount to send and let the function calculate this amount and transfer it from the user.

Status

Resolved

Medium

[M-01] PinGameLock#initialize - No checks for duplicate items in the array can result in setting incorrect prices for tokens

Description

The `initialize` function initializes the contract by adding inputted addresses as supported tokens and assigning price values to these addresses. However, a lack of checks for duplicate addresses in this array can lead to setting an addresses price value more than once, leading to assigning incorrect values.

Vulnerability Details

The `initialize` function initializes the contract by adding inputted addresses as supported tokens and assigning price values to these addresses.

```
function initialize(address _owner,uint256[] memory _prices,address[] memory
_supportedTokenList)public initializer {

    // require(_price > 0,"price should be greater than 0");

    supportedTokenList = _supportedTokenList;

    for (uint i = 0; i < supportedTokenList.length;i++){

        supportedTokens[supportedTokenList[i]] = true;

        originPrices[supportedTokenList[i]] = _prices[i];

    }

    // price = _price;

    __Ownable_init(_owner);

}
```

As observed, there are no checks done to make sure that the address in the for loop has not been assigned a price already. This allows the same token address being assigned a price value more than once. Accidental inputs where the same token address is present more than once in the array will lead to assigning incorrect price values for this address.

Impact

Accidental inputs allow assigning incorrect price values for token addresses.

Recommendation

Implement checks to make sure a token address is not present more than once in the inputted array. An example is shown below.

```
function initialize(address _owner,uint256[] memory _prices,address[] memory
_supportedTokenList)public initializer {

    // require(_price > 0,"price should be greater than 0");

    supportedTokenList = _supportedTokenList;

    for (uint i = 0; i < supportedTokenList.length;i++){

        require(!supportedTokens[supportedTokenList[i]], "Duplicate token address");

        supportedTokens[supportedTokenList[i]] = true;

        originPrices[supportedTokenList[i]] = _prices[i];

    }

    // price = _price;

    __Ownable_init(_owner);

}
```

Status

Resolved

[M-02] PinGameLock#refundByOwner - Supported token being removed after users have made a deposit with this token leads to refunds not working for these users

Description

The `refundByOwner` function allows the owner to refund users who have deposited in the contract. This function implements a check to make sure that the inputted address is a supported token. In edge cases where users have deposited with a token but this token is removed after the `removeSupportedToken` function, these users will not be able to be refunded.

Vulnerability Details

The `refundByOwner` function allows the owner to refund users who have deposited in the contract.

```
function refundByOwner(address _token,address _user, uint256 _amount)onlyOwner
external{

    require(supportedTokens[_token] ,"token is not supported");

    require(userBalance[_user][_token] >= _amount,"balance is not enough");

    userBalance[_user][_token] -= _amount;

    if (userBalance[_user][_token] == 0){

        userBought[_user]=false;

    }

    SafeERC20.safeTransfer(IERC20(_token),_user,_amount);

    emit Refund(_token,_user,_amount);

}
```

As observed in this function, the highlighted check makes sure that the token inputted is a supported token. However, this check is unnecessary as the following line already checks if the user has deposited using this token.

Taking a look at the `removeSupportedToken` function



```

function removeSupportedToken(address tokenAddress) onlyOwner external {

    require(supportedTokens[tokenAddress], "token not exist");

    supportedTokens[tokenAddress] = false;

    for (uint i = 0; i < supportedTokenList.length; i++) {

        if (supportedTokenList[i] == tokenAddress) {

            supportedTokenList[i] = supportedTokenList[supportedTokenList.length - 1];

            supportedTokenList.pop();

            break;

        }

    }

    emit RemoveTokenEvent(tokenAddress);

}

```

It is observed that, without any additional checks to see if users have deposited with this function, it will remove the supported token. When a supported token is removed this way, users will no longer be able to be refunded by the owner.

Impact

Users will not be able to be refunded by the owner if the token they deposited with has been removed from the supported tokens.

Recommendation

Remove the check in the function that checks if the token inputted is a supported token.

Status

Resolved

[M-03] PinGameLock - Contract does not remove fully used discounts leading them to accumulate over time and increasing the array size which can lead to bricking the contract due to an out of gas error

Description

When a discount is added with the `setDiscount` function, this function sorts the array so the greatest discount amount is the first item in the array. However, fully used discounts are not removed from the array at any point. This will lead to fully used discounts accumulating in the contract over time, increasing the gas cost of the `updateCurrentDiscount` calls. The array can grow infinitely which can even lead to bricking `deposit` and `setDiscount` calls permanently due to OoG errors.

Vulnerability Details

A discount can be added for a token with the `setDiscount` function

```
function setDiscount(address _token,uint256 discount_price,uint256 discount_amount)
onlyOwner external {

    require(discount_price > 0,"price should be greater than 0");

    require(discount_amount >= 0,"amount should be greater than 0");

    require(supportedTokens[_token],"token not exist");

    // sort the discounts array descending order

    discountsAmounts[_token].push(discount_price);

    // console.log(discountsAmounts[_token][0]);

    uint256[] memory sortedAmounts =
Arrays.sort(discountsAmounts[_token],Comparators.gt);

    discountsAmounts[_token] = sortedAmounts;

    discounts[_token][discount_price] = discount_amount;

    updateCurrentDiscount(_token);

    emit SetDiscount(_token, discount_price, discount_amount);

}
```

As observed in this function, it will sort the array so that the greatest discount is the first item in the array and will call the `updateCurrentDiscount` function.

```
function updateCurrentDiscount(address _token) internal {  
    uint256[] memory amounts = discountsAmounts[_token];  
    for (uint i = 0; i < amounts.length; i++){  
        if (discounts[_token][amounts[i]] > 0){  
            currentDiscount[_token] = amounts[i];  
            return ;  
        }  
    }  
    currentDiscount[_token] = 0;  
}
```

As observed, this function will loop through the array until it finds a discount that still has uses left and set this as the `currentDiscount`. Since there is no functionality in the contracts to remove fully used discounts, the array will grow infinitely, increasing the gas cost of `updateCurrentDiscount` calls which will lead to reverts due to an OoG error.

Impact

Gas costs for the `updateCurrentDiscount` calls will keep growing over time. The cost of calling this function can increase enough to cause reverts due to an OoG error which will brick the contract functionality.

Recommendation

Implement new functionality to remove fully used discounts.

Status

Resolved

Low

[L-01] PinGameLock - Contract lacks `_disableInitializers`

Description

`_disableInitializers()` should be used in the implementation contract to stop it from being initialized. Initializing the implementation contract by calling the `initialize` function can have unintended consequences especially if it is done by malicious actors. OpenZeppelin's warning for this issue can be seen below.

⚠ CAUTION

Avoid leaving a contract uninitialized.

An uninitialized contract can be taken over by an attacker. This applies to both a proxy and its implementation contract, which may impact the proxy. To prevent the implementation contract from being used, you should invoke the `_disableInitializers` function in the constructor to automatically lock it when it is deployed:

```
/// @custom:oz-upgrades-unsafe-allow constructor
constructor() {
    _disableInitializers();
}
```

Recommendation

Implement `_disableInitializers` in the constructor.

Status

Resolved

[L-02] PinGameLock - Imported contracts are unused

Description

The imported contracts shown below are not used in the contract and should be removed or be used in the contract.

```
import "@openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/utils/ReentrancyGuardUpgradeable.sol";
import "@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import "@openzeppelin/contracts/utils/Pausable.sol";
import "@openzeppelin/contracts/utils/math/Math.sol";
```

Recommendation

Remove or make use of the unused inherited contracts.

Status

Resolved

[L-03] PinGameLock#initialize - Lack of `address(0)` checks for inputted addresses in the initializer

Description

The `initialize` function lacks `address(0)` checks to prevent potential mistakes. It is good practice to implement `address(0)` checks in the `initialize` function.

Recommendation

Implement `address(0)` checks in the `initialize` function. An example is shown below.

```
function initialize(address _owner,uint256[] memory _prices,address[] memory
_supportedTokenList)public initializer {

    // require(_price > 0,"price should be greater than 0");

    supportedTokenList = _supportedTokenList;

    for (uint i = 0; i < supportedTokenList.length;i++){
        require(supportedTokenList[i] != address(0), "0 address");

        supportedTokens[supportedTokenList[i]] = true;
```

```

        originPrices[supportedTokenList[i]] = _prices[i];

    }

    // price = _price;

    __Ownable_init(_owner);

}

```

Status

Resolved

[L-04] PinGameLock#initialize - No checks done to make sure array lengths match leaves contract vulnerable to accidental inputs

Description

The `initialize` function initializes the contract by adding inputted addresses as supported tokens and assigning price values to these addresses. However, there are no checks done to make sure that the inputted array lengths match. This leaves the contract vulnerable to accidental inputs.

Recommendation

Implement a check in the function to make sure array lengths match. An example is shown below.

```

function initialize(address _owner, uint256[] memory _prices, address[] memory
_supportedTokenList) public initializer {

    require(_prices.length == _supportedTokenList.length, "Arrays length mismatch");

    // rest of the function
}

```

Status

Resolved

[L-05] PinGameLock#setDiscountValidTo, setPrice - Prevent setting a state variable with the same value

Description

Not only is it wasteful in terms of gas, but this is especially problematic when an event is emitted and the old and new values set are the same, as listeners might not expect this kind of scenario.

Recommendation

Implement checks in the functions to prevent this scenario. An example is shown below.

```
function setDiscountValidTo(uint256 _validTo) onlyOwner external {  
    require(discountValidTo != _validTo, "same value");  
  
    discountValidTo = _validTo;  
}
```

Status

Resolved

[L-06] Contracts - Use of floating pragma

Description

Contracts use a pragma statement that does not specify a fixed compiler version but instead allows the contract to be compiled with any compatible compiler version. This can lead to various compatibility and stability issues.

Recommendation

Consider locking the pragma version whenever possible and avoid using a floating pragma in the final deployment. Consider [known bugs](#) for the compiler version that is chosen.

Status

Acknowledged

[L-07] **PinGameLock#setDiscountValidTo,** **withdrawAllByOwner,** **withdrawByOwner** - Important functions lack event emissions

Description

Critical functions should emit events to provide transparency and enable off-chain monitoring of important state changes. Without events, it becomes difficult to track and verify contract operations, complicating both user interfaces and protocol monitoring.

Recommendation

Implement event emissions for critical state variable updates in the function.

Status

Resolved

[L-08] **PinGameLock** - Upgradeable contract is missing `__gap[...]` storage variable

Description

Storage gaps are needed to not break storage layouts when adding new variables to base contracts. Listed contracts may not be inherited but it is good practice to add them now rather than forgetting later.

Recommendation

When designing upgradeable contracts, it's important to include `__gap[...]` storage variables as storage gaps to prevent issues with storage layout changes when adding new variables to base contracts in the future. While the listed contracts may not be directly inherited, it is a good practice to add `__gap[...]` storage variables now to ensure a robust and future-proof upgradeable contract design. This proactive approach helps avoid potential complications and storage layout conflicts in the later stages of contract development.

Status

Resolved

[L-09] PinGameLock - Use Ownable2Step versions rather than Ownable versions

Description

Ownable2StepUpgradeable prevents the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using Ownable2StepUpgradeable from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Acknowledged

[L-10] PinGameLock#addSupportedToken - Token price is not added when a new token is supported

Description

The addSupportedToken function allows the owner to add a new token to be supported and used in the contracts. However, this function does not set a price for the token that is being added which will not allow this token to be used until setPrice is called for it.

Recommendation

Change the function to set the tokens price. An example is shown below.

```
function addSupportedToken(address _token, uint256 _price) onlyOwner external {
```

```

    require(!supportedTokens[_token], "already added");

    require(_price > 0, "must be greater than 0");

    supportedTokens[_token] = true;

    originPrices[_token] = _price;

    supportedTokenList.push(_token);

    emit AddSupportedToken(_token);

    emit SetPrice(_token, 0, _price);
}

```

Status

Resolved

[L-11] PinGameLock#addSupportedToken - Lack of address(0) check

Description

The `addSupportedToken` function allows the owner to add a new token to be supported and used in the contracts. This function lacks to check if the inputted token address is `address(0)`.

Recommendation

Implement checks for `address(0)`. An example is shown below.

```

function addSupportedToken(address _token, uint256 _price) onlyOwner external {

    require(!supportedTokens[_token], "already added");

    require(_token != address(0), "zero address");

    // rest of the function
}

```

Status

Resolved

[L-12] PinGameLock#prices - Lack of 0 checks

Description

These functions lack 0 checks. This allows accidental inputs to:

- Break tokens use if price is set to 0
- Stop discounts from being used if discount valid to is set to 0

Recommendation

Implement 0 checks in these functions. An example is shown below.

```
function prices(address _token) external view returns(uint256) {  
    require(supportedTokens[_token], "token not exist");  
    require(originPrices[_token] > 0, "can't be 0");  
    return originPrices[_token] - getCurrentDiscount(_token);  
}
```

Status

Resolved

[L-13] PinGameLock - Centralization risk for privileged functions

Description

Privileged functions need the privileged role to be trusted not to perform malicious updates. This may also cause a single point failure. Functions with the centralization risk are shown below.

```
function setDiscountValidTo()  
  
function setPrice()
```

```
function addSupportedToken()  
  
function removeSupportedToken()  
  
function setDiscount()  
  
function withdrawAllByOwner()  
  
function withdrawByOwner()  
  
function refundByOwner()
```

Recommendation

To mitigate centralization risks associated with privileged functions, consider implementing a multi-signature or decentralized governance mechanism. Instead of relying solely on a single owner/administrator, distribute control and decision-making authority among multiple parties or stakeholders.

Status

Acknowledged

Gas

[G-01] PinGameLock#removeSupportedToken - Function should remove data associated with the token being removed

Description

The `removeSupportedToken` function allows the owner to remove a token from the supported tokens. However, even when a token is removed, the associated data with this token persists in the contract.

Recommendation

When removing a token from the supported tokens, remove all associated data with this token such as its price and discounts.

Status

Acknowledged

[G-02] PinGameLock#initialize, removeSupportedToken, updateCurrentDiscount, withdrawAllByOwner - Cache array length before looping

Description

Before iterating through an array with a for-loop, the length of the array can be cached in memory so that its value does not need to be accessed on each iteration of the loop.

Recommendation

Cache the array length by assigning it to a variable in memory before looping through the array.

Status

Acknowledged

[G-03] **PinGameLock#initialize, removeSupportedToken, updateCurrentDiscount, withdrawAllByOwner** - Use unchecked for loop increments when overflow is impossible

Description

Solidity version 0.8.22 introduces an overflow check optimization that automatically generates an unchecked arithmetic increment of the counter of for loops. Since the contracts use 0.8.20, this optimization does not exist and causes waste of gas.

Recommendation

Move the for loop increments into the loop body and make the increments unchecked. Alternatively, use Solidity version 0.8.22 for this optimization.

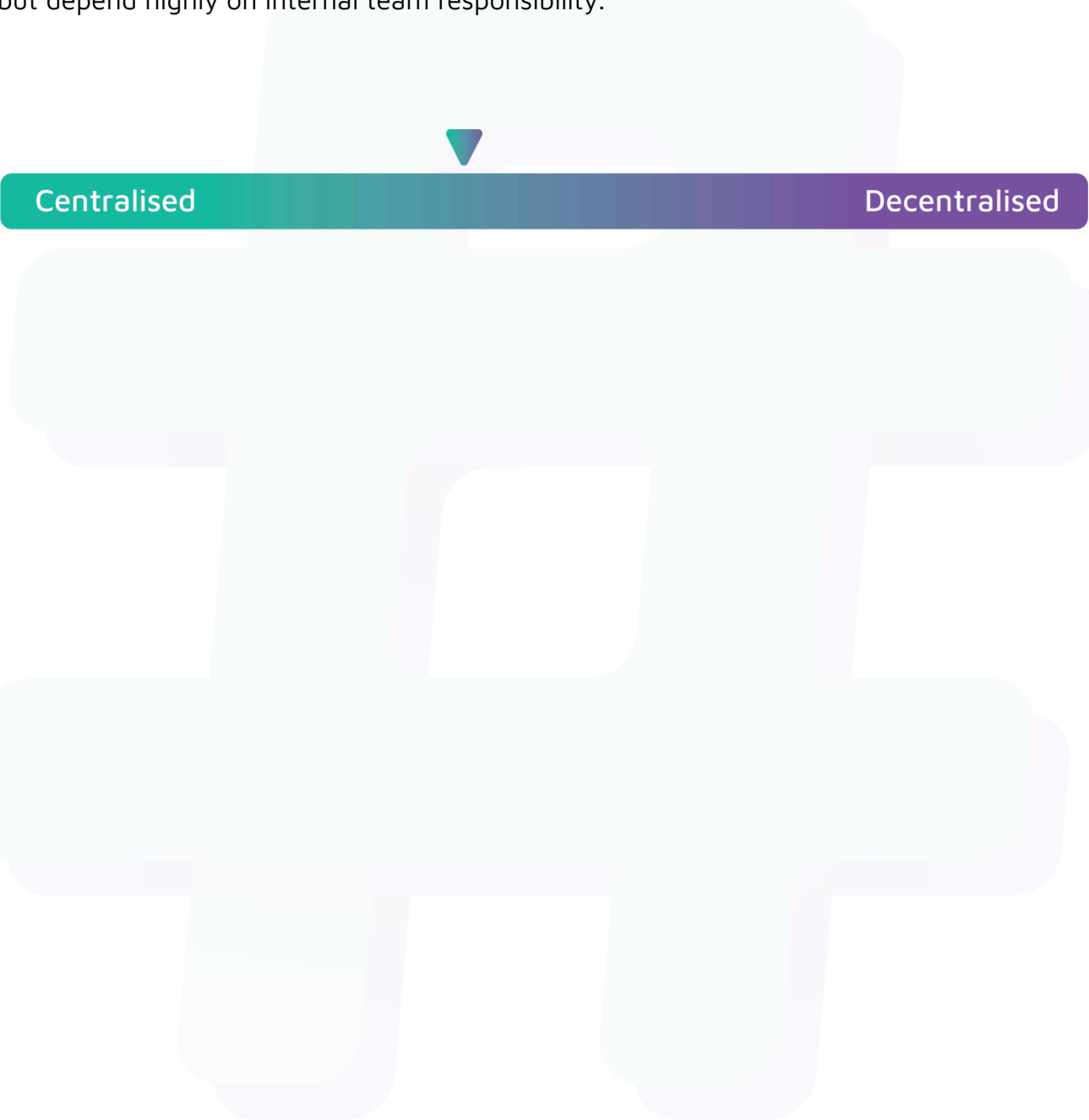
Status

Resolved

Centralisation

The PING project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the PING project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.